Principios de Arquitectura de Software · Retos Software III

Enrutamiento inteligente con Service Mesh

Ruteo por cabeceras HTTP con Linkerd sobre Kubernetes · Reto 4

Kubernetes | Linkerd | Gateway API | kind

David Esteban Suarez Lozano · 2221909
Thomas Alejandro Perez Rojas · 2221906
Jesus David Santodomingo Carrascal · 2221922

Agenda

1. **Contexto**
   El problema y lo que pide el reto
2. **Service Mesh**
   Cómo funciona Linkerd y por qué HTTPRoute
3. **Arquitectura**
   Qué desplegamos en el clúster
4. **Implementación**
   Scripts, app y regla de ruteo
5. **Pruebas**
   curl con y sin cabecera
6. **Observabilidad**
   Métricas, mTLS y experimentos

1 · Contexto

Controlar el tráfico entre versiones de un microservicio

El problema

1 Pruebas A/B

Mostrar una variante solo a un grupo de usuarios

2 Canary

Liberar una versión nueva a una fracción del tráfico

3 Feature flags

Activar una funcionalidad con una cabecera

4 Seguridad y métricas

mTLS y observabilidad entre servicios

OBJETIVO DEL RETO

Petición	Versión
X-Feature-Flag: new-version	v2
User-Agent: experimental	v2
Sin cabecera u otro valor	v1

…sin modificar el código de la aplicación

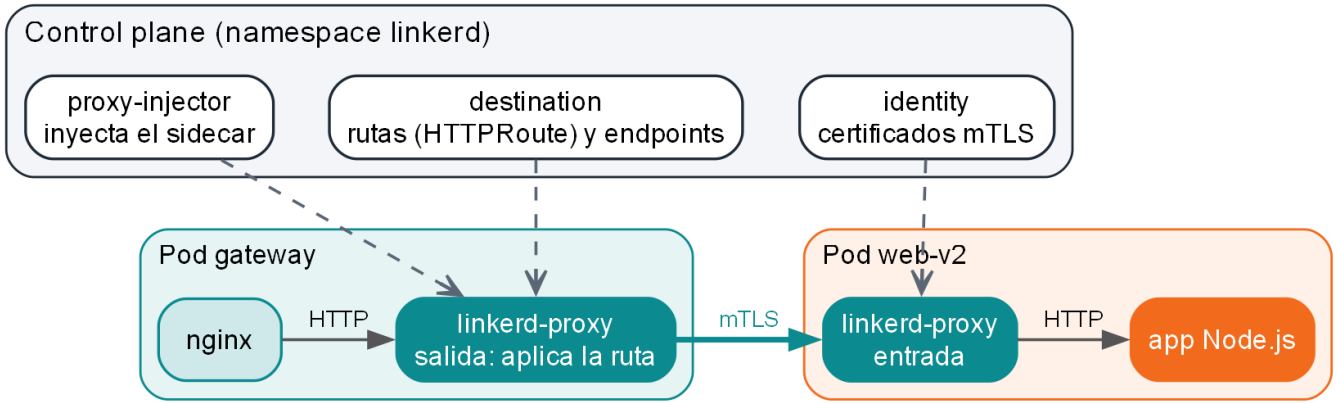
Lo que pide el reto y cómo lo cumplimos

Requisito del enunciado	Cómo lo resolvimos
App desplegada en Kubernetes con un Service Mesh	Clúster kind + Linkerd edge-26.10.1
Reglas de ruteo con CRDs del mesh	HTTPRoute (Gateway API) web-por-cabecera
Cabecera → versión específica	x-feature-flag / user-agent → web-v2
Lo demás → versión estable	Regla por defecto → web-v1
Verificar con curl o Postman	Script de 100 peticiones + colección Postman
Explorar la observabilidad del mesh	viz stat, stat-outbound, edges, tap, dashboard
Discutir Service Mesh vs Ingress	Comparación al cierre

2 · Service Mesh y Linkerd

Una capa de red transparente para la aplicación

Cómo funciona Linkerd



Inyección

Anotación linkerd.io/inject: enabled → cada pod recibe linkerd-init y linkerd-proxy (sidecar nativo)

Proxy en Rust

Intercepta el tráfico con iptables; la app sigue hablando HTTP normal

mTLS + métricas

Cifra pod a pod con identidad propia y mide éxito, RPS y latencia

HTTPRoute en lugar de TrafficSplit

TrafficSplit + ServiceProfile

SMI (deprecado en Linkerd)

– Solo reparte tráfico por pesos

– No puede leer cabeceras

– Especificación SMI archivada

HTTPRoute

Gateway API · GAMMA (recomendado)

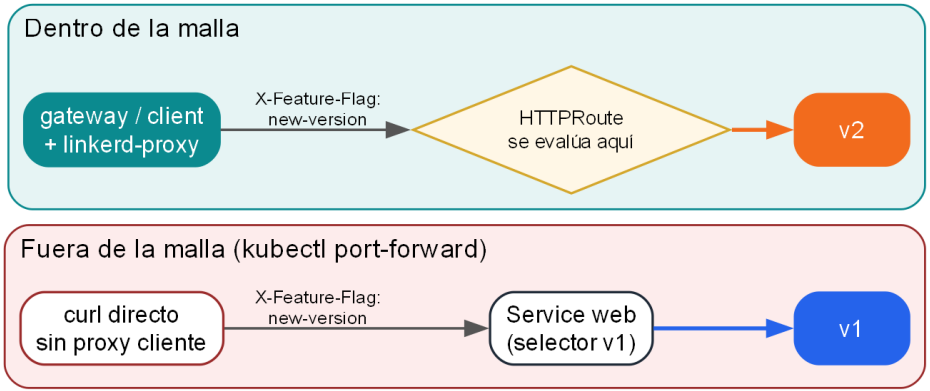
+ matches por cabecera, ruta, método o query

+ Pesos para canary en la misma regla

+ Estándar de Kubernetes: sirve también en Istio

El enunciado menciona TrafficSplit/ServiceProfile; usamos su reemplazo oficial: el parentRef de la ruta es un Service, no un Gateway

El ruteo ocurre en el proxy cliente

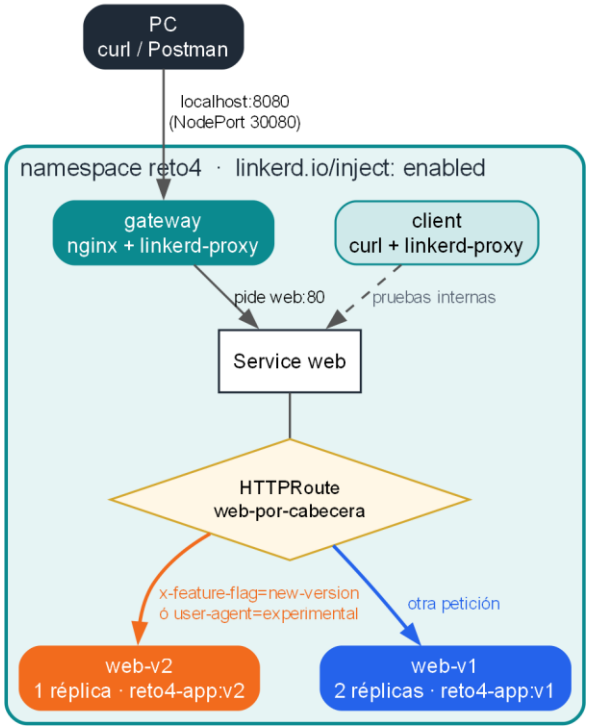


- Linkerd aplica el HTTPRoute en el proxy de quien envía la petición
- Una petición que no sale por un linkerd-proxy no se enruta
- Por eso pusimos un gateway nginx dentro de la malla como entrada desde el PC
- **Lo comprobamos: por port-forward, 5/5 respuestas fueron v1 aun con la cabecera**

3 · Arquitectura

Qué desplegamos en el clúster

Arquitectura de la solución



1. **gateway**
   nginx mallado: entrada desde el PC por localhost:8080

2. **Service web**
   Nombre que usan los clientes; su selector apunta solo a v1

3. **HTTPRoute**
   Decide v1 o v2 según la cabecera

4. **web-v1 / web-v2**
   Misma imagen con APP_VERSION distinto (2 y 1 réplicas)

5. **client**
   Pod mallado con curl para pruebas internas

Services: seguro por defecto

k8s/20-services.yaml (resumido)

```
apiVersion: v1
kind: Service
metadata: { name: web, namespace: reto4 }
spec:
  selector: { app: web, version: v1 }  # solo v1
  ports: [{ port: 80, targetPort: http }]
---
# backends que referencia el HTTPRoute
kind: Service
metadata: { name: web-v1 }
spec: { selector: { app: web, version: v1 } }
---
kind: Service
metadata: { name: web-v2 }
spec: { selector: { app: web, version: v2 } }
```

1. **Un nombre estable**

   Los clientes siempre llaman a web; no saben que hay versiones

2. **Un Service por versión**

   web-v1 y web-v2 son los destinos de la ruta

3. **Falla segura**

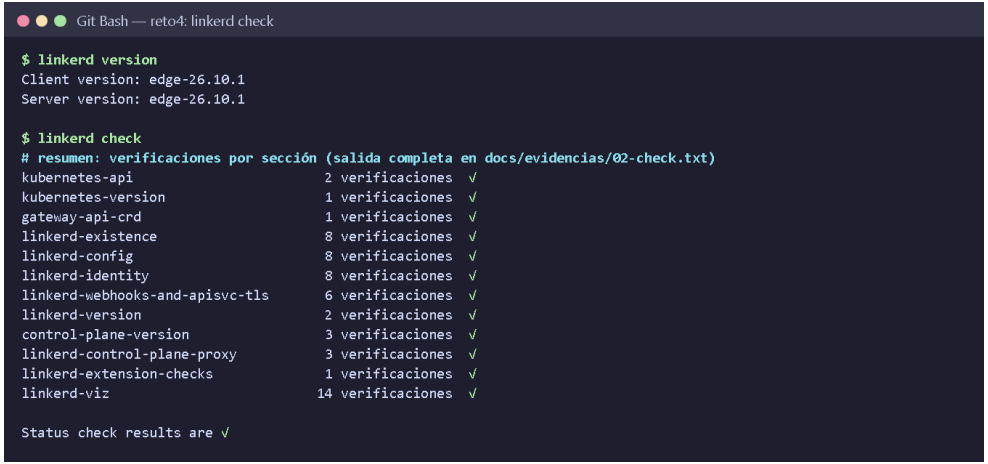
   Si se borra el HTTPRoute, todo vuelve a v1

4 · Implementación

De cero a ruteo por cabeceras con scripts

Entorno y herramientas

Componente	Versión
Docker Desktop · Windows 11	29.9.1
kind (nodo kindest/node)	0.33.0 · K8s 1.37.0
kubectl	1.36.1
Linkerd + viz	edge-26.10.1
Gateway API CRDs	v1.2.1 standard
Node.js / nginx / curl	22 / 1.27 / 8.11



linkerd check: 57 verificaciones en verde

Paso a paso

1

Clúster

kind con el puerto 8080 publicado

2

Linkerd

Gateway API CRDs, control plane y viz

3

App

Build v1/v2, kind load y manifiestos

4

Ruta

kubectl apply del HTTPRoute

5

Pruebas

20 peticiones por caso y conteo

Git Bash · raíz del repositorio

```
scripts/01-crear-cluster.sh
scripts/02-instalar-linkerd.sh
scripts/03-desplegar-app.sh
scripts/04-aplicar-rutas.sh header
scripts/05-probar.sh 20
scripts/06-observabilidad.sh
```

Todo es reproducible: scripts/99-limpiar.sh borra el clúster y se vuelve a levantar en ~10 minutos

La aplicación: una imagen, dos versiones

app/server.js (resumido)

```
const VERSION = process.env.APP_VERSION || 'v1';

http.createServer((req, res) => {
  const body = {
    version: VERSION,
    pod: os.hostname(),
    cabeceras: {
      'x-feature-flag': req.headers['x-feature-flag'] || null,
      'user-agent': req.headers['user-agent'] || null } };
  res.writeHead(200, { 'X-App-Version': VERSION });
  res.end(JSON.stringify(body, null, 2));
}).listen(8080);
```

Dockerfile con ARG APP_VERSION

```
docker build -t reto4-app:v1 --build-arg APP_VERSION=v1 app
docker build -t reto4-app:v2 --build-arg APP_VERSION=v2 app
```

- Node.js sin dependencias externas
- Responde qué versión y qué pod atendió
- Devuelve las cabeceras recibidas: así vemos lo que llegó
- **La app no sabe nada de ruteo: no tiene un solo if por cabecera**

La regla: HTTPRoute por cabecera

linkerd/httproute-header.yaml

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: { name: web-por-cabecera, namespace: reto4 }
spec:
  parentRefs:
    - { name: web, kind: Service, group: core, port: 80 }
  rules:
    - matches:                          # OR entre matches
        - headers: [{ name: x-feature-flag, value: new-version }]
        - headers: [{ name: user-agent,     value: experimental }]
      backendRefs: [{ name: web-v2, port: 80 }]
    - backendRefs: [{ name: web-v1, port: 80 }]   # por defecto
```

1 **parentRef = Service web**

La regla se aplica al tráfico dirigido a web

2 **Regla 1 → v2**

Cualquiera de las dos cabeceras (nombre sin distinguir mayúsculas, valor exacto)

3 **Regla 2 → v1**

Sin matches: atrapa todo lo demás

5 · Pruebas

curl con y sin cabecera: 20 peticiones por caso

Antes vs después del HTTPRoute

Git Bash — reto4: pruebas sin HTTPRoute

```
$ scripts/05-probar.sh 20   # ANTES de aplicar el HTTPRoute

==> Desde el PC (localhost:8080 -> gateway mallado -> web), 20 peticiones por caso
Sin cabecera                              v1=20  v2=0   error=0
X-Feature-Flag: new-version               v1=20  v2=0   error=0
X-Feature-Flag: otro-valor                v1=20  v2=0   error=0
User-Agent: experimental                  v1=20  v2=0   error=0
x-feature-flag (en minusculas)            v1=20  v2=0   error=0
```

ANTES: la cabecera no cambia nada, todo va a v1

Git Bash — reto4: pruebas con HTTPRoute

```
$ scripts/05-probar.sh 20   # DESPUES de aplicar el HTTPRoute

==> Desde el PC (localhost:8080 -> gateway mallado -> web), 20 peticiones por caso
Sin cabecera                              v1=20  v2=0   error=0
X-Feature-Flag: new-version               v1=0   v2=20  error=0
X-Feature-Flag: otro-valor                v1=20  v2=0   error=0
User-Agent: experimental                  v1=0   v2=20  error=0
x-feature-flag (en minusculas)            v1=0   v2=20  error=0
```

DESPUÉS: la cabecera lleva el tráfico a v2

Lo que demuestra

Misma app y mismas cabeceras en las dos pruebas.

El único cambio es el HTTPRoute: el desvío lo hace Linkerd, no el código.

Resultados

100/100

peticiones llegaron a la versión esperada

0 errores · desde el PC y desde el pod cliente

Caso (20 peticiones c/u)	Esperado	v1	v2	
Sin cabecera	v1	20	0	✓
X-Feature-Flag: new-version	v2	0	20	✓
X-Feature-Flag: otro-valor	v1	20	0	✓
User-Agent: experimental	v2	0	20	✓
x-feature-flag (minúsculas)	v2	0	20	✓

También probamos casos negativos: otro valor de la cabecera sigue yendo a v1

Una petición por dentro

Terminal

```
$ curl -i -H "X-Feature-Flag: new-version"
http://localhost:8080/
HTTP/1.1 200 OK
x-app-version: v2

{
  "version": "v2",
  "mensaje": "Nueva version (feature flag activo)",
  "pod": "web-v2-5bcff59747-gnzbz",
  "cabeceras": { "x-feature-flag": "new-version",
                 "user-agent": "curl/8.16.0" }
}
```

x-app-version

Cabecera de respuesta con la versión que atendió

pod

Nombre real del pod: web-v2-…

Postman

Las mismas 4 peticiones en docs/reto4.postman_collection.json

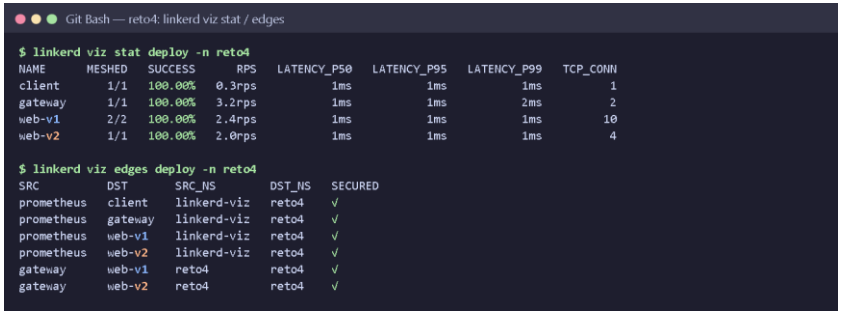
6 · Observabilidad

Linkerd viz: métricas sin instrumentar la aplicación

Métricas por ruta, mTLS y tráfico en vivo



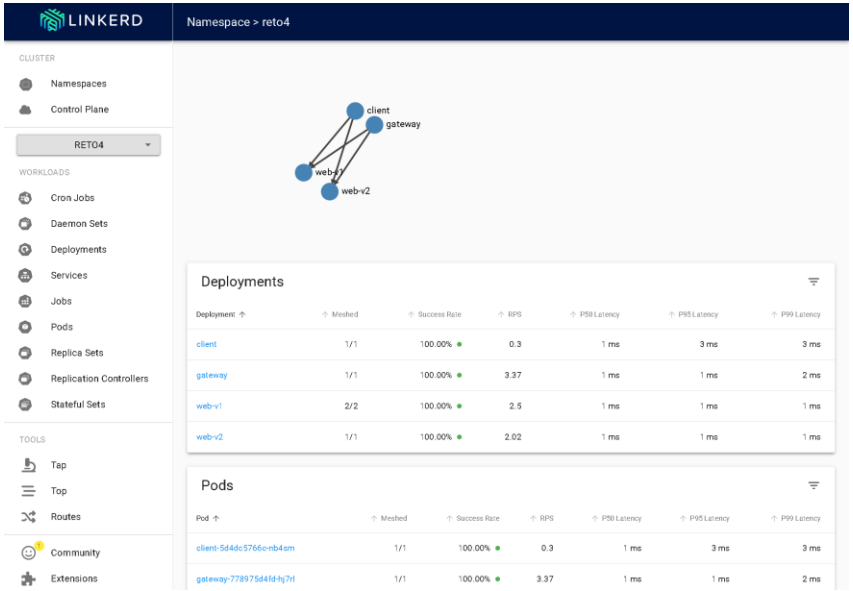
stat-outbound: la ruta reparte 1,55 RPS a cada versión · tap: tls=true



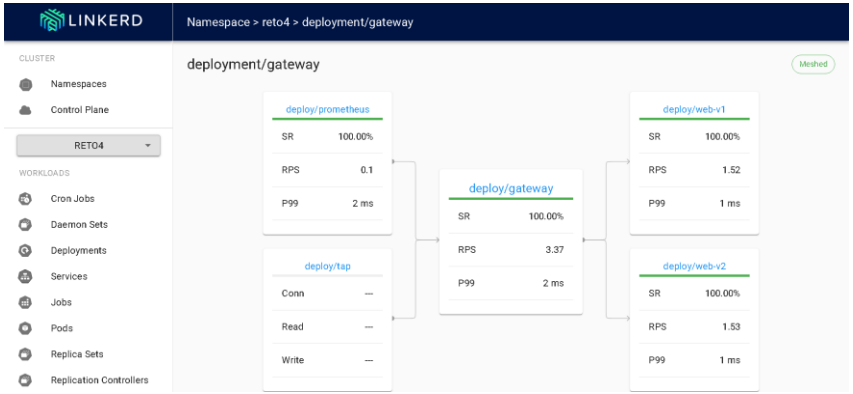
stat: 100 % de éxito · edges: gateway → v1/v2 con SECURED √

- stat-outbound ve el tráfico desde el cliente: ruta y backend
- tap: llegan a v2 peticiones dirigidas a web.reto4.svc
- edges: todas las conexiones con mTLS automático
- Latencia p99 de 1–2 ms: el proxy agrega muy poco

Dashboard de Linkerd viz



Namespace reto4: grafo de tráfico y métricas por deployment

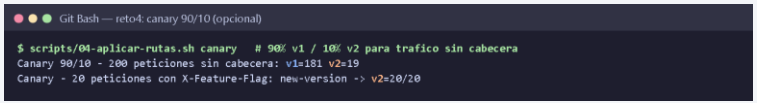


deployment/gateway: el tráfico sale hacia web-v1 y web-v2

Experimentos adicionales

Canary 90/10 + cabecera

181 / 19 de 200 peticiones sin cabecera fueron a v1 / v2



Las 20 con cabecera fueron a v2: feature flag para testers y canary gradual para el resto

Fuera de la malla

5 / 5 respuestas de v1 aun enviando la cabecera



Sin proxy cliente no se aplica la ruta: confirma dónde decide Linkerd

Service Mesh vs Ingress vs balanceador

Balanceador L4

+ Simple y barato
+ Muy bajo overhead

– No entiende HTTP: sin cabeceras
– Sin mTLS ni métricas por servicio

Ingress (borde)

+ Ruteo HTTP para tráfico externo
+ TLS hacia el usuario

– No ve llamadas entre servicios
– TLS termina en el borde

Service Mesh

+ Reglas en cada llamada, también este-oeste
+ mTLS y métricas automáticas

– Un proxy por pod (CPU/RAM)
– Más componentes que operar

Retos técnicos

1

Herramientas fuera del PATH

env.sh agrega Docker, kind y linkerd a cada script

2

El proxy no aparecía en los pods

Linkerd 2026 usa sidecar nativo: está en initContainers

3

La cabecera no cambiaba nada

El ruteo es del proxy cliente: gateway nginx mallado

4

TrafficSplit deprecado

Migramos al estándar: HTTPRoute de Gateway API

Conclusiones

- Una cabecera HTTP decide la versión sin tocar el código: 100/100 peticiones correctas
- El diseño falla de forma segura: sin HTTPRoute todo vuelve a v1
- Linkerd da mTLS y métricas por ruta sin instrumentar la aplicación
- En Linkerd el ruteo vive en el proxy cliente: quien llama debe estar en la malla
- La misma ruta con pesos combina feature flags y canary con riesgo controlado